UNIVERSITÉ DE VERSAILLES
SAINT-QUENTIN-EN-YVELINES

IUT de Vélizy – BUT R&T 3e année

# SAE du Hacker

## Documentation des challenges CrackMe

Désassemblage & Reverse Engineering

**Adam, Mathéo, Dawson, Théo**

Année universitaire 2025 – 2026

Mars 2026

# Table des matières


**1 Challenge 1** . . . **3**
- 1.1 Mots de passe possibles . . . 3
- 1.2 Démarche effectuée . . . 3
  - 1.2.1 Observation initiale . . . 3
  - 1.2.2 Analyse avec `strings` et `radare2` . . . 3
  - 1.2.3 Confirmation par désassemblage . . . 3
- 1.3 Réécriture du code en C . . . 4

**2 Challenge 2** . . . **5**
- 2.1 Mots de passe possibles . . . 5
- 2.2 Démarche effectuée . . . 5
  - 2.2.1 Observation initiale . . . 5
  - 2.2.2 Analyse avec `strings` . . . 5
  - 2.2.3 Confirmation par désassemblage . . . 5
- 2.3 Réécriture du code en C . . . 5

**3 Challenge 3** . . . **7**
- 3.1 Mots de passe possibles . . . 7
- 3.2 Démarche effectuée . . . 7
  - 3.2.1 Observation initiale . . . 7
  - 3.2.2 Recherche des chaînes . . . 7
  - 3.2.3 Où sont cachées les chaînes ? . . . 7
  - 3.2.4 Explication technique . . . 8
- 3.3 Réécriture du code en C . . . 8

**4 Challenge 4** . . . **9**
- 4.1 Mots de passe possibles . . . 9
- 4.2 Démarche effectuée . . . 9
  - 4.2.1 Observation initiale . . . 9
  - 4.2.2 Analyse de la section .data . . . 9
  - 4.2.3 Confirmation par désassemblage . . . 9
- 4.3 Réécriture du code en C . . . 10

**5 Challenge 5** . . . **11**
- 5.1 Mots de passe possibles . . . 11
- 5.2 Démarche effectuée . . . 12
  - 5.2.1 Observation initiale du programme . . . 12
  - 5.2.2 Analyse avec `strings` . . . 12
  - 5.2.3 Identification du type d'entrée . . . 12
  - 5.2.4 Analyse de la section .data . . . 12
  - 5.2.5 Confirmation par désassemblage . . . 13
- 5.3 Réécriture du code en C . . . 13

**6 Challenge 6** . . . **15**
- 6.1 Mots de passe possibles . . . 15
- 6.2 Démarche effectuée . . . 15
  - 6.2.1 Structure du programme . . . 15
  - 6.2.2 Recherche de constantes . . . 15
  - 6.2.3 Analyse du désassemblage . . . 16
- 6.3 Réécriture du code en C . . . 16

**7 Challenge 7** . . . **17**
- 7.1 Mots de passe possibles . . . 17

7.2 Démarche effectuée . . . 17
7.2.1 Observation initiale . . . 17
7.2.2 Analyse avec `radare2` . . . 18
7.2.3 Analyse du `main` . . . 18
7.2.4 Analyse de `check_pal` . . . 19
7.2.5 Confirmation avec GDB . . . 20
7.3 Réécriture du code en C . . . 21

**8 Challenge 8 . . . 23**
8.1 Mots de passe possibles . . . 23
8.2 Démarche effectuée . . . 23
8.2.1 Observation initiale . . . 23
8.2.2 Analyse avec radare2 . . . 24
8.2.3 Analyse du `main` . . . 25
8.2.4 Analyse de `check_ma` . . . 25
8.2.5 Analyse de `check_mi` . . . 25
8.3 Réécriture du code en C . . . 25

# 1 Challenge 1

## 1.1 Mots de passe possibles

Le programme accepte une seule combinaison valide :

| Champ | Valeur |
|---|---|
| Login | `superuser` |
| Password | `AlphaPass` |

Exemple d'exécution valide :

```
$ ./crackmes_1
 ---------------------
 ----- CRACKME 1 -----
 ---------------------
Login : superuser
Password : AlphaPass
** ACCESS GRANTED **
```

## 1.2 Démarche effectuée

### 1.2.1 Observation initiale

On commence par lancer le programme. Celui-ci demande un login et un mot de passe de manière interactive via `scanf`. Si les valeurs sont incorrectes, il affiche `** ACCESS DENIED **`.

### 1.2.2 Analyse avec `strings` et `radare2`

On utilise d'abord la commande `strings` pour extraire les chaînes lisibles du binaire. On repère tout de suite deux chaînes suspectes : `superuser` et `AlphaPass`.

Pour confirmer, on ouvre le binaire dans `radare2` et on utilise la commande `iz` pour lister les chaînes de la section `.rodata` :

```
$ r2 crackmes_1
[0x00001090]> iz
[Strings]
0 0x00002008  9 10 .rodata ascii superuser
1 0x00002012  9 10 .rodata ascii AlphaPass
2 0x0000201c 23 24 .rodata ascii ----------------------
...
```

Les deux chaînes `superuser` et `AlphaPass` sont stockées en clair dans la section `.rodata`. Elles correspondent au login et au mot de passe attendus.

### 1.2.3 Confirmation par désassemblage

Le désassemblage de la fonction `main` montre que le programme utilise deux appels à `strcmp` pour comparer les entrées de l'utilisateur avec les valeurs de référence. La logique est simple :

1. Lecture du login avec `scanf("%s", ...)`
2. Lecture du password avec `scanf("%s", ...)`
3. Comparaison du login saisi avec "`superuser`" via `strcmp`

4. Si le login est bon, comparaison du password avec `ÄlphaPass¨` via `strcmp`
5. Si les deux correspondent : `ACCESS GRANTED`

C'est le challenge le plus simple : les identifiants sont directement visibles dans le binaire sans aucune obfuscation. Un simple `strings crackmes_1` suffit.

### 1.3 Réécriture du code en C

```c
#include <stdio.h>
#include <string.h>

char *login = "superuser";
char *passw = "AlphaPass";

int main(void)
{
    char input_login[30];
    char input_passw[30];

    puts(" ---------------------");
    puts(" ----- CRACKME 1 -----");
    puts(" ---------------------\n");

    printf("Login : ");
    scanf("%s", input_login);
    printf("Password : ");
    scanf("%s", input_passw);

    if (strcmp(login, input_login) == 0 &&
        strcmp(passw, input_passw) == 0) {
        puts("** ACCESS GRANTED **");
    } else {
        puts("** ACCESS DENIED **");
    }
    return 0;
}
```

# 2 Challenge 2

## 2.1 Mots de passe possibles

| Champ | Valeur |
|---|---|
| Login (argv[1]) | `du6Thoo7` |
| Password (argv[2]) | `EW8ingoh` |

Les identifiants sont passés en arguments de la ligne de commande.
Exemple d'exécution valide :

```
$ ./crackmes_2 du6Thoo7 EW8ingoh
 ---------------------
 ----- CRACKME 2 -----
 ---------------------
** ACCESS GRANTED **
```

## 2.2 Démarche effectuée

### 2.2.1 Observation initiale

En lançant le programme sans arguments, on obtient le message `Missing login and password !`. On comprend donc que le login et le password ne sont pas saisis de manière interactive : ils doivent être passés en arguments de la commande (`argv[1]` et `argv[2]`).
Le désassemblage confirme que le programme vérifie `argc == 3` avant de continuer.

### 2.2.2 Analyse avec `strings`

On utilise `strings` sur le binaire et on repère deux chaînes qui ressemblent à des identifiants :

```
$ strings crackmes_2
...
du6Thoo7
EW8ingoh
...
```

Ces chaînes sont stockées dans la section `.rodata`, exactement comme pour le challenge 1.

### 2.2.3 Confirmation par désassemblage

Le désassemblage montre la même logique que le challenge 1, sauf que les entrées proviennent de `argv` au lieu de `scanf`. Le programme récupère `argv[1]` et `argv[2]`, puis les compare avec les chaînes de référence via `strcmp`.

## 2.3 Réécriture du code en C

```c
#include <stdio.h>
#include <string.h>

char *login = "du6Thoo7";
```

```c
char *passw = "EW8ingoh";

int main(int argc, char *argv[])
{
    puts(" ---------------------");
    puts(" ----- CRACKME 2 -----");
    puts(" ---------------------\n");

    if (argc != 3) {
        puts("Missing login and password !");
        return 1;
    }
    if (strcmp(login, argv[1]) == 0 &&
        strcmp(passw, argv[2]) == 0) {
        puts("** ACCESS GRANTED **");
    } else {
        puts("** ACCESS DENIED **");
    }
    return 0;
}
```

# 3 Challenge 3

## 3.1 Mots de passe possibles

| Champ | Valeur |
|---|---|
| Login (argv[1]) | `XahHaf1J` |
| Password (argv[2]) | `Fae9v` |

Exemple d'exécution valide :

```
$ ./crackmes_3 XahHaf1J Fae9v
 ---------------------
 ----- CRACKME 3 -----
 ---------------------
** ACCESS GRANTED **
```

## 3.2 Démarche effectuée

### 3.2.1 Observation initiale

Le comportement est identique au challenge 2 : le programme attend deux arguments en ligne de commande et affiche `Missing login and password !` sinon.

### 3.2.2 Recherche des chaînes

On lance `strings` sur le binaire. On voit bien les chaînes `XahHaf1J` et `Fae9v` apparaître. Cependant, si on utilise la commande `iz` dans `radare2`, ces chaînes **n'apparaissent pas**, car elles ne sont pas dans la section `.rodata` ni dans `.data`.

Pour les trouver, il faut utiliser `izz` (recherche dans **toutes** les sections).

### 3.2.3 Où sont cachées les chaînes ?

En inspectant la table des symboles avec `nm`, on découvre que les symboles `login` et `passw` sont des symboles **locaux** dans la section `.text` (le code exécutable) :

```
$ nm crackmes_3 | grep -E "login|passw"
000011a0 t login
000011ac t passw
```

Le `t` minuscule indique un symbole local dans la section texte. En dumpant la mémoire à ces adresses, on retrouve nos chaînes :

```
$ objdump -s --start-address=0x11a0 --stop-address=0x11b2 -j .text
   crackmes_3
 11a0 58616848 6166314a 008d7600 46616539  XahHaf1J..v.Fae9
 11b0 7600                                 v.
```

C'est le piège de ce challenge : le compilateur a placé les variables `login` et `passw` dans la section `.text` au lieu de `.data`. La commande `iz` de radare2 ne suffit pas, il faut utiliser `izz` ou inspecter la table des symboles.

### 3.2.4 Explication technique

Quand on déclare une variable `static` locale à un fichier dans certaines conditions, le compilateur peut choisir de la placer dans la section `.text` plutôt que `.data`. C'est ce qui s'est passé ici avec les déclarations de type :

```
static char login[] = "XahHaf1J";
static char passw[] = "Fae9v";
```

## 3.3 Réécriture du code en C

```
#include <stdio.h>
#include <string.h>

/* Declarees en static pour reproduire le placement en .text */
static char login[] = "XahHaf1J";
static char passw[] = "Fae9v";

int main(int argc, char *argv[])
{
    puts(" ---------------------");
    puts(" ----- CRACKME 3 -----");
    puts(" ---------------------\n");

    if (argc != 3) {
        puts("Missing login and password !");
        return 1;
    }
    if (strcmp(login, argv[1]) == 0 &&
        strcmp(passw, argv[2]) == 0) {
        puts("** ACCESS GRANTED **");
    } else {
        puts("** ACCESS DENIED **");
    }
    return 0;
}
```

# 4 Challenge 4

## 4.1 Mots de passe possibles

| Champ | Valeur décimale | Valeur hexadécimale |
|---|---|---|
| Login (argv[1]) | 77 | 0x4D |
| Password (argv[2]) | 99 | 0x63 |

Les valeurs sont des **entiers**, pas des chaînes de caractères. Elles sont passées en arguments. Exemple d'exécution valide :

```
$ ./crackmes_4 77 99
 ---------------------
 ----- CRACKME 4 -----
 ---------------------
** ACCESS GRANTED **
```

## 4.2 Démarche effectuée

### 4.2.1 Observation initiale

En testant avec des chaînes de caractères, le programme affiche `Incorrect login type`. On en déduit que les identifiants ne sont pas des strings mais plutôt des entiers.

```
$ ./crackmes_4 test testpass
 ---------------------
 ----- CRACKME 4 -----
 ---------------------
Incorrect login type
```

En testant avec des entiers aléatoires, on obtient bien `ACCESS DENIED` au lieu du message d'erreur de type. Le programme utilise donc `atoi()` pour convertir les arguments.

### 4.2.2 Analyse de la section .data

On inspecte la section `.data` pour trouver les constantes de référence :

```
$ objdump -s -j .data crackmes_4
 400c 00000000 10400000 4d000000 63000000
```

Les valeurs sont stockées en **little-endian 32 bits**. On identifie :

| Adresse | Hex (little-endian) | Décimal |
|---|---|---|
| 0x4014 (login) | `4D 00 00 00` | 77 |
| 0x4018 (passw) | `63 00 00 00` | 99 |

### 4.2.3 Confirmation par désassemblage

Le désassemblage de `main` confirme la logique :

1. Vérification de `argc == 3`

2. Conversion de `argv[1]` et `argv[2]` en entiers via `atoi()`
3. Si `atoi` retourne 0, le type est considéré invalide
4. Comparaison directe des entiers avec les valeurs en `.data` (77 et 99)

Contrairement aux challenges précédents, ici on utilise `cmp` directement sur des entiers, pas `strcmp`. La présence de `atoi` dans les imports (`strings` ou `rabin2 -i`) est un indice important.

### 4.3 Réécriture du code en C

```c
#include <stdio.h>
#include <stdlib.h>

int login = 77;
int passw = 99;

int main(int argc, char *argv[])
{
    puts(" ---------------------");
    puts(" ----- CRACKME 4 -----");
    puts(" ---------------------\n");

    if (argc != 3) {
        puts("No login password set !");
        return 1;
    }

    int l = atoi(argv[1]);
    if (l == 0) {
        puts("Incorrect login type");
        return 2;
    }

    int p = atoi(argv[2]);
    if (p == 0) {
        puts("Incorrect password type");
        return 3;
    }

    if (l == login && p == passw) {
        puts("** ACCESS GRANTED **");
    } else {
        puts("** ACCESS DENIED **");
    }
    return 0;
}
```

# 5 Challenge 5

## 5.1 Mots de passe possibles

Le programme accepte une seule combinaison valide :

| Champ | Valeur décimale | Valeur hexadécimale |
|---|---|---|
| Login | 145 | 0x91 |
| Password | 954 | 0x3BA |

Les valeurs doivent être saisies au format entier lors de l'exécution interactive du programme. Exemple d'exécution valide :

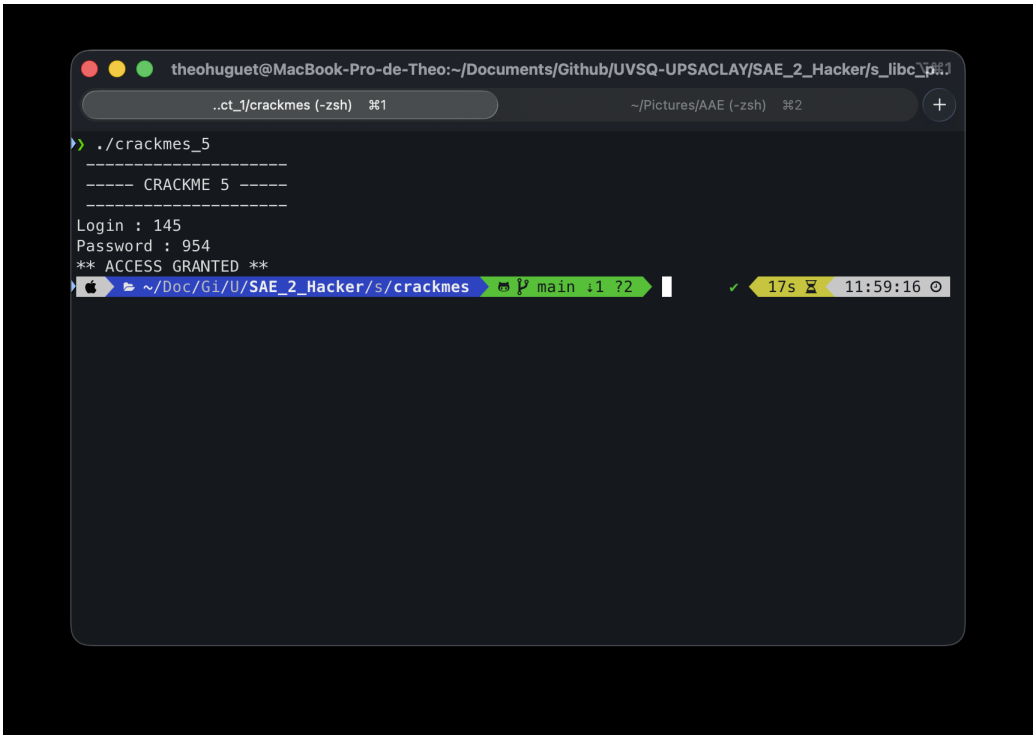


FIGURE 1 – Exécution de crackmes_5 avec les bonnes valeurs

Toute autre combinaison provoque l'affichage de `** ACCESS DENIED **` :

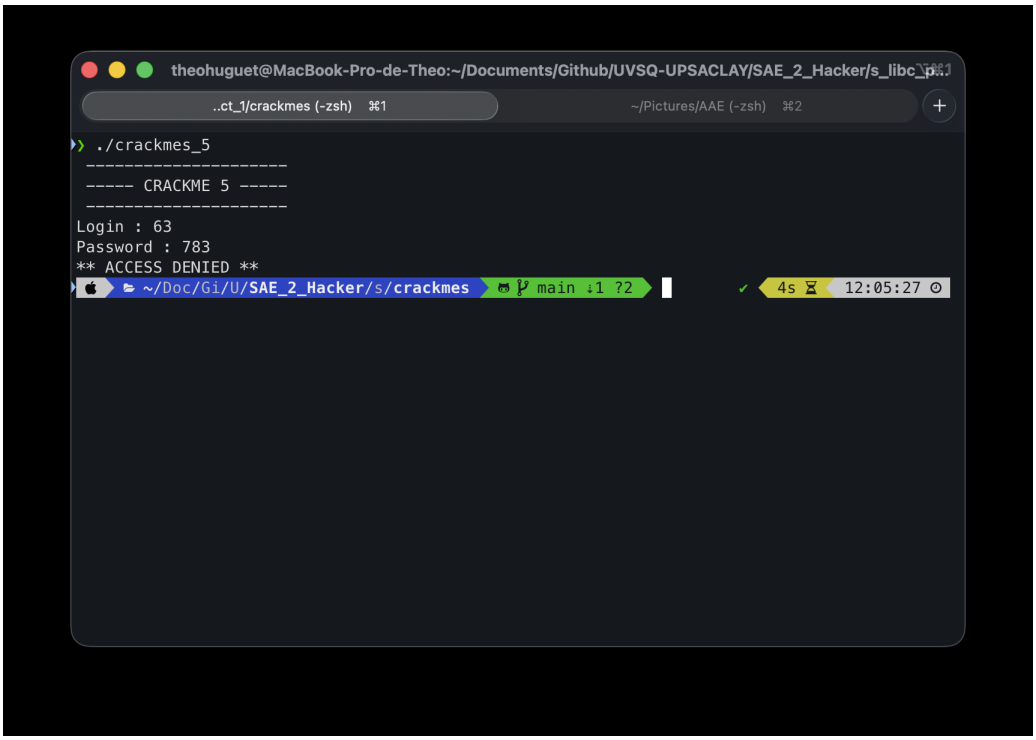


FIGURE 2 – Exécution de crackmes_5 avec des valeurs incorrectes

## 5.2 Démarche effectuée

### 5.2.1 Observation initiale du programme

Lors de l'exécution du programme, celui-ci affiche les messages `Login :` et `Password :`. Cela indique que les identifiants sont saisis de manière interactive, contrairement aux challenges 2, 3 et 4.

### 5.2.2 Analyse avec `strings`

L'outil `strings` permet d'extraire les chaînes présentes dans le binaire. On identifie notamment les messages `Incorrect login type` et `Incorrect password type`, ce qui suggère l'utilisation d'une fonction de lecture typée.

### 5.2.3 Identification du type d'entrée

L'analyse révèle l'utilisation de `scanf("%d", ...)`. Cela signifie que les entrées sont des entiers. Si `scanf` ne parvient pas à convertir la valeur saisie en entier, la fonction renvoie une valeur différente de 1, et le programme détecte un type de saisie invalide.

### 5.2.4 Analyse de la section .data

On inspecte la section `.data` du binaire :

```
$ objdump -s -j .data crackmes_5
 4014: 00000000 18400000 91000000 ba030000
```

Ces valeurs sont stockées en little-endian 32 bits. Après conversion :

| Valeur hexadécimale | Valeur décimale |
|---|---|
| 0x00000091 | 145 |
| 0x000003BA | 954 |

### 5.2.5 Confirmation par désassemblage

Le désassemblage de la fonction `main` confirme que ces valeurs sont utilisées dans des comparaisons directes :

```
mov edx, [ebx+0x28]   ; login_ref = 145
mov eax, [ebp-0x10]   ; login saisi
cmp edx, eax
jne ACCESS_DENIED

mov edx, [ebx+0x2c]   ; pass_ref = 954
mov eax, [ebp-0x14]   ; password saisi
cmp edx, eax
jne ACCESS_DENIED
```

## 5.3 Réécriture du code en C

```c
#include <stdio.h>

static const int login_ref = 0x91;  /* 145 */
static const int pass_ref = 0x3BA;  /* 954 */

int main(void)
{
    int login, password, res;

    puts("---------------------");
    puts("----- CRACKME 5 -----");
    puts("---------------------");

    printf("Login : ");
    fflush(stdout);
    res = scanf("%d", &login);
    if (res != 1) {
        puts("Incorrect login type");
        return 1;
    }

    printf("Password : ");
    fflush(stdout);
    res = scanf("%d", &password);
    if (res != 1) {
        puts("Incorrect password type");
        return 2;
    }

    if (login == login_ref && password == pass_ref) {
        puts("** ACCESS GRANTED **");
    } else {
        puts("** ACCESS DENIED **");
```

```
    }
    return 0;
}
```

# 6 Challenge 6

## 6.1 Mots de passe possibles

Contrairement aux challenges précédents, il n'existe pas de combinaison unique. Toute paire d'entiers où `login == password` est acceptée.

Exemples valides :

| Login | Password | Résultat |
|---|---|---|
| 42 | 42 | ACCESS GRANTED |
| 1000 | 1000 | ACCESS GRANTED |
| -7 | -7 | ACCESS GRANTED |
| 87 | 76 | ACCESS DENIED |

Le programme accepte donc une **infinité de solutions**.

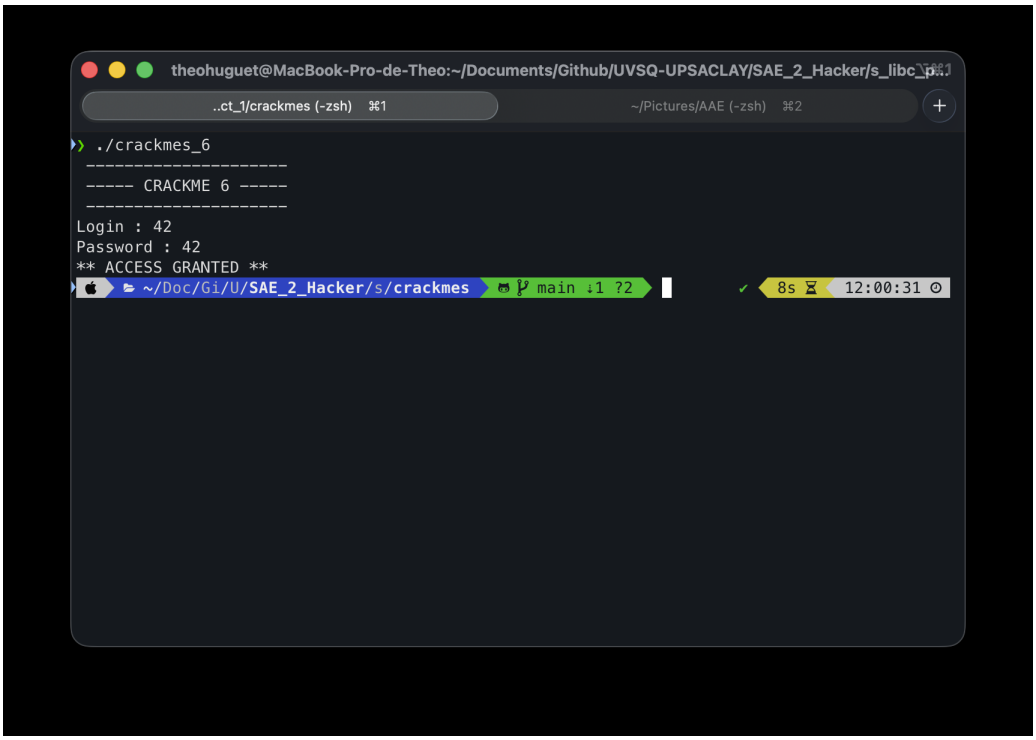


FIGURE 3 – Exécutions de crackmes_6 – succès et échec

## 6.2 Démarche effectuée

### 6.2.1 Structure du programme

La structure est similaire au challenge 5 : interface interactive avec lecture des entrées via `scanf("%d")`.

### 6.2.2 Recherche de constantes

On inspecte les chaînes (`strings`) et la section `.data`. Contrairement au challenge précédent, **aucune constante numérique pertinente** n'est trouvée. Cela suggère que la validation ne dépend pas d'une valeur fixe.

### 6.2.3 Analyse du désassemblage

L'analyse de la fonction `main` met en évidence la séquence suivante :

```
mov edx, [ebp-0x10]    ; login saisi
mov eax, [ebp-0x14]    ; password saisi
cmp edx, eax
jne ACCESS_DENIED
```

Contrairement au challenge 5, aucune constante n'est chargée depuis `.data`. Les deux opérandes proviennent directement de la pile. Le programme compare simplement les deux entrées entre elles.

Nombre de solutions : **infini**. Toute paire d'entiers identiques fonctionne. La « loi » évoquée dans le sujet est tout simplement l'égalité `login == password`.

## 6.3 Réécriture du code en C

```c
#include <stdio.h>

int main(void)
{
    int login, password, res;

    puts("---------------------");
    puts("----- CRACKME 6 -----");
    puts("---------------------");

    printf("Login : ");
    fflush(stdout);
    res = scanf("%d", &login);
    if (res != 1) {
        puts("Incorrect login type");
        return 1;
    }

    printf("Password : ");
    fflush(stdout);
    res = scanf("%d", &password);
    if (res != 1) {
        puts("Incorrect password type");
        return 2;
    }

    if (login == password) {
        puts("** ACCESS GRANTED **");
    } else {
        puts("** ACCESS DENIED **");
    }
    return 0;
}
```

# 7 Challenge 7

## 7.1 Mots de passe possibles

Le CrackMe 7 n'a pas un mot de passe unique. Le programme accepte n'importe quel mot de passe qui est un **palindrome**, c'est-à-dire un mot qui se lit de la même façon de gauche à droite et de droite à gauche.

| Mot de passe | Résultat |
|---|---|
| `kayak` | ACCESS GRANTED |
| `radar` | ACCESS GRANTED |
| `level` | ACCESS GRANTED |
| `aba` | ACCESS GRANTED |
| `racecar` | ACCESS GRANTED |
| `test` | ACCESS DENIED |
| `hello` | ACCESS DENIED |

Syntaxe : `./crackmes_7 <mot_de_passe>`. Seul le mot de passe est utilisé (`argv[1]`).

```
user@debian:~/Documents$ ./crackmes_7 kayak
 ---------------------
 ----- CRACKME 7 -----
 ---------------------

** ACCESS GRANTED **
```

Figure 4 – Exécution de crackmes_7 avec un palindrome

## 7.2 Démarche effectuée

### 7.2.1 Observation initiale

On commence par lancer le programme sans argument :

```
user@debian:~/Documents$ ./crackmes_7
 ---------------------
 ----- CRACKME 7 -----
 ---------------------

Missing  password !
```

Figure 5 – crackmes_7 sans argument – message d'erreur

Le programme affiche le bandeau `CRACKME 7` puis le message `Missing password !`. On comprend donc qu'il faut passer le mot de passe en argument de la commande.

### 7.2.2 Analyse avec `radare2`

On ouvre le binaire dans `radare2` et on lance l'analyse automatique avec `aaa`. En listant les fonctions détectées avec `afl`, on repère deux fonctions intéressantes : `main` à l'adresse `0x00001208` et `check_pal` à l'adresse `0x0000119d`. On remarque aussi que le programme utilise les fonctions de la libc `puts` et `strlen`.

```
[0x00001070]> afl
0x00001030    1      6 sym.imp.__libc_start_main
0x00001040    1      6 sym.imp.puts
0x00001050    1      6 sym.imp.strlen
0x00001070    1     39 entry0
0x00001098    1      4 fcn.00001098
0x000010b0    4     52 sym.deregister_tm_clones
0x00001199    1      4 sym.__x86.get_pc_thunk.dx
0x000010f0    4     71 sym.register_tm_clones
0x00001140    5     75 entry.fini0
0x000010a0    1      4 sym.__x86.get_pc_thunk.bx
0x00001060    1      6 fcn.00001060
0x00001190    1      9 entry.init0
0x000012d0    1     20 sym._fini
0x00001208    7    195 main
0x000012cb    1      4 sym.__x86.get_pc_thunk.ax
0x0000119d    7    107 sym.check_pal
0x00001000    3     32 sym._init
```

FIGURE 6 – Liste des fonctions détectées par radare2

### 7.2.3 Analyse du `main`

On désassemble le `main` avec `pdf @main`. On y retrouve les trois appels à `puts` pour l'affichage du bandeau, puis une vérification du nombre d'arguments (`cmp dword [esi], 2`). Si l'utilisateur n'a pas donné d'argument, le programme affiche `Missing password !` et quitte. Sinon, le mot de passe est récupéré et passé à la fonction `check_pal`. Si cette fonction retourne 1, le programme affiche `ACCESS GRANTED`, sinon `ACCESS DENIED`.

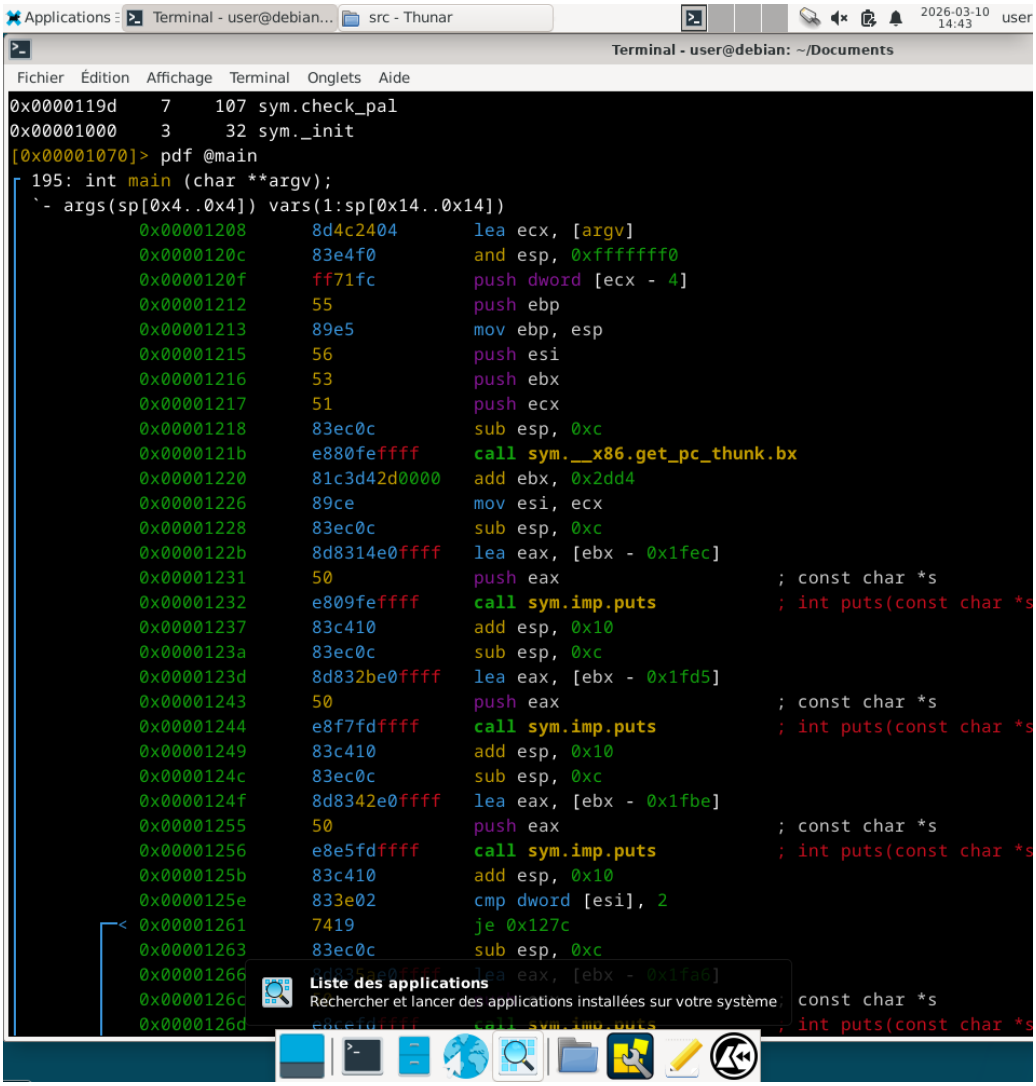


FIGURE 7 – Désassemblage de `main` dans radare2

### 7.2.4 Analyse de `check_pal`

La partie clé est dans `check_pal`. On la désassemble avec `pdf @ 0x0000119d`. La fonction fait les choses suivantes :

1. Elle appelle `strlen` pour calculer la longueur du mot de passe
2. Elle initialise deux compteurs : un au début de la chaîne (position 0) et un à la fin (position longueur – 1)
3. Elle entre dans une boucle qui compare le caractère à la position début avec le caractère à la position fin
4. Si les deux caractères sont différents, la fonction retourne 0 (échec)
5. Si les deux caractères sont identiques, elle avance le compteur début de 1 et recule le compteur fin de 1
6. La boucle continue tant que le compteur début est inférieur au compteur fin
7. Si tous les caractères ont correspondu, la fonction retourne 1 (succès)

Ce comportement correspond exactement à la vérification classique d'un **palindrome** : on compare les caractères symétriques en partant des extrémités vers le centre.

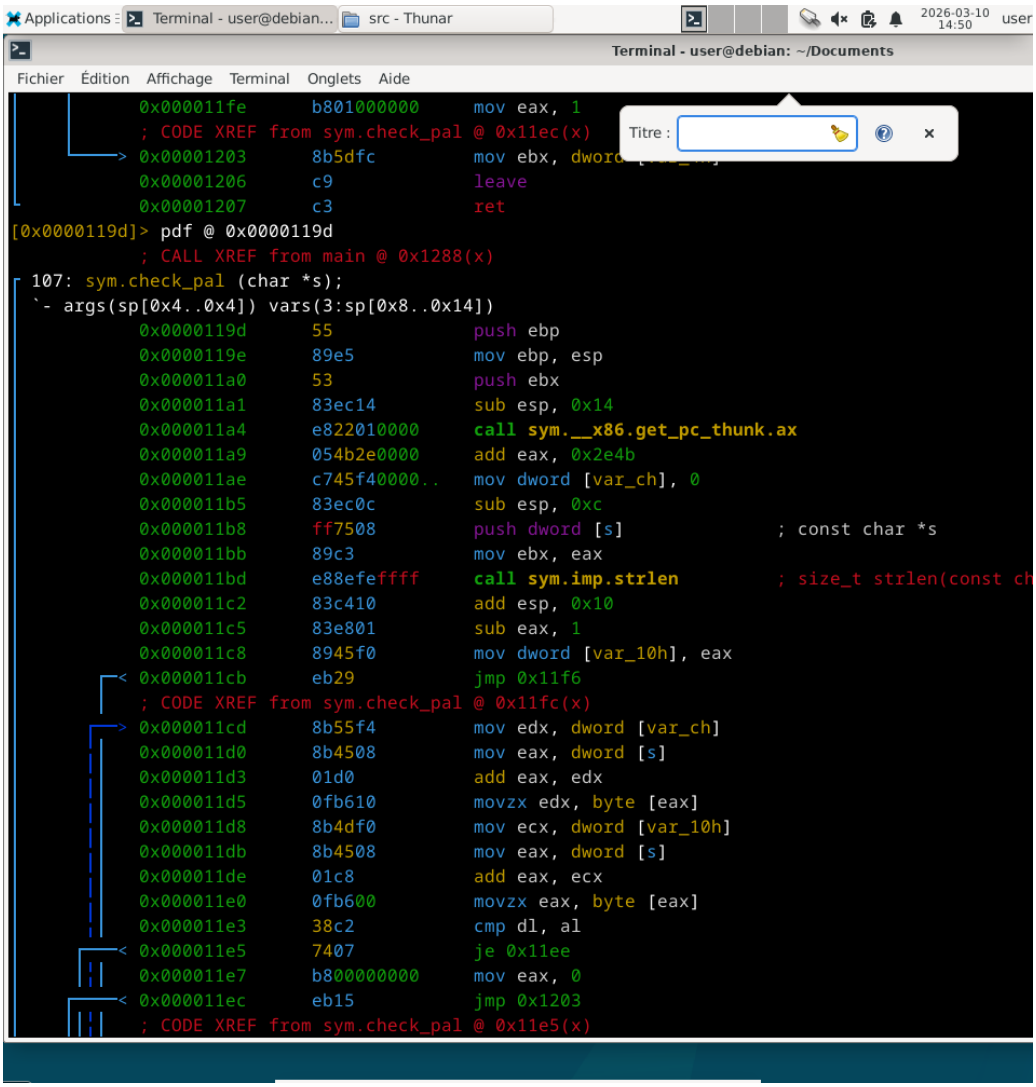


FIGURE 8 – Désassemblage de `check_pal` dans radare2

### 7.2.5 Confirmation avec GDB

Pour confirmer notre analyse, on utilise aussi GDB pour observer le programme en cours d'exécution :

```
gdb ./crackmes_7
break check_pal
run kayak
info registers
```

Le programme se fige bien à l'entrée de la fonction `check_pal`. On peut observer dans les registres que le programme est en train de traiter notre chaîne "`kayak`".

```
Breakpoint 1, 0x565561a1 in check_pal ()
(gdb) info registers
eax            0xffffd389          -11383
ecx            0xf7e1e9b8          -136189512
edx            0x1                 1
ebx            0x56558ff4          1448447988
esp            0xffffd0c4          0xffffd0c4
ebp            0xffffd0c8          0xffffd0c8
esi            0xffffd110          -12016
edi            0xf7ffcb80          -134231168
eip            0x565561a1          0x565561a1 <check_pal+4>
eflags         0x296               [ PF AF SF IF ]
cs             0x23                35
ss             0x2b                43
ds             0x2b                43
es             0x2b                43
fs             0x0                 0
gs             0x63                99
(gdb)
```

FIGURE 9 – Breakpoint dans `check_pal` avec GDB

### 7.3 Réécriture du code en C

À partir de l'analyse du code assembleur, on peut reconstituer le programme original en C :

```c
#include <stdio.h>
#include <string.h>

int check_pal(char *s)
{
    int debut = 0;
    int fin = strlen(s) - 1;

    while (debut < fin) {
        if (s[debut] != s[fin]) {
            return 0;
        }
        debut++;
        fin--;
    }
    return 1;
}

int main(int argc, char *argv[])
{
    puts("----------------------");
    puts("----- CRACKME 7 -----");
    puts("----------------------");

    if (argc != 2) {
        puts("Missing  password !");
        return 1;
```

```c
    }

    if (check_pal(argv[1]) == 1) {
        puts("** ACCESS GRANTED **");
    } else {
        puts("** ACCESS DENIED **");
    }
    return 0;
}
```

On a compilé et testé ce code pour vérifier qu'il produit les mêmes résultats que le binaire original :

```
user@debian:~/Documents$ mousepad ~/Documents/crackme7.c
user@debian:~/Documents$ gcc -m32 -o mon_crackme7 crackme7.c
user@debian:~/Documents$ ./mon_crackme7 kayak
----------------------
----- CRACKME 7 -----
----------------------
** ACCESS GRANTED **
user@debian:~/Documents$ ./mon_crackme7 test
----------------------
----- CRACKME 7 -----
----------------------
** ACCESS DENIED **
user@debian:~/Documents$
```

Figure 10 – Compilation et test du code reconstitué

Le comportement est identique, ce qui confirme que notre réécriture est fidèle au programme original.

# 8 Challenge 8

## 8.1 Mots de passe possibles

Tout mot de passe contenant **autant de lettres majuscules que de lettres minuscules** fonctionne. Les chiffres et caractères spéciaux sont ignorés dans le comptage.

| Mot de passe | Résultat |
|---|---|
| `Aa` | ACCESS GRANTED |
| `AaAa` | ACCESS GRANTED |
| `AAaa` | ACCESS GRANTED |
| `""` (vide) | ACCESS GRANTED |
| `11` (chiffres seuls) | ACCESS GRANTED |
| `test` | ACCESS DENIED |
| `TEST` | ACCESS DENIED |

Syntaxe : `./crackmes_8 <mot_de_passe>`. Seul le mot de passe est utilisé (`argv[1]`).

```
user@debian:~/Téléchargements$ ./crackmes_8 Aa
 ---------------------
 ----- CRACKME 8 -----
 ---------------------

** ACCESS GRANTED **
user@debian:~/Téléchargements$ ./crackmes_8 AaAa
 ---------------------
 ----- CRACKME 8 -----
 ---------------------

** ACCESS GRANTED **
user@debian:~/Téléchargements$ ./crackmes_8 ""
 ---------------------
 ----- CRACKME 8 -----
 ---------------------

** ACCESS GRANTED **
user@debian:~/Téléchargements$ ./crackmes_8 11
 ---------------------
 ----- CRACKME 8 -----
 ---------------------

** ACCESS GRANTED **
user@debian:~/Téléchargements$ ./crackmes_8 1
 ---------------------
 ----- CRACKME 8 -----
 ---------------------

** ACCESS GRANTED **
```

FIGURE 11 – Différents tests de crackmes_8

## 8.2 Démarche effectuée

### 8.2.1 Observation initiale

On commence par observer le comportement du programme sans l'analyser. On voit qu'il attend un mot de passe en argument et qu'il affiche soit `ACCESS GRANTED`, soit `ACCESS DENIED`.

```
user@debian:~/Téléchargements$ ./crackmes_8
 ---------------------
 ----- CRACKME 8 -----
 ---------------------

Missing  password !
user@debian:~/Téléchargements$ ./crackmes_8 test
 ---------------------
 ----- CRACKME 8 -----
 ---------------------

** ACCESS DENIED **
user@debian:~/Téléchargements$ ./crackmes_8 Aa
 ---------------------
 ----- CRACKME 8 -----
 ---------------------

** ACCESS GRANTED **
```

FIGURE 12 – Tests initiaux de crackmes_8

### 8.2.2 Analyse avec radare2

On lance l'analyse automatique avec `aaa`. L'analyse des chaînes avec `iz` ne révèle rien de particulier en dehors des messages d'affichage classiques.

```
user@debian:~/Téléchargements$ r2 ./crackmes_8
WARN: Relocs has not been applied. Please use `-e bin.relocs.apply=true` or `-e bin.cache=true` next time
 -- I can't believe you came back.
[0x00001060]> aaa
INFO: Analyze all flags starting with sym. and entry0 (aa)
INFO: Analyze imports (af@@@i)
INFO: Analyze entrypoint (af@ entry0)
INFO: Analyze symbols (af@@@s)
INFO: Analyze all functions arguments/locals (afva@@F)
INFO: Analyze function calls (aac)
INFO: Analyze len bytes of instructions for references (aar)
INFO: Finding and parsing C++ vtables (avrr)
INFO: Analyzing methods (af @@ method.*)
INFO: Recovering local variables (afva@@@F)
INFO: Type matching analysis for all functions (aaft)
INFO: Propagate noreturn information (aanr)
INFO: Use -AA or aaaa to perform additional experimental analysis
```

FIGURE 13 – Chaînes extraites avec `iz`

En revanche, en listant les fonctions avec `afl`, on repère deux fonctions au nom très évocateur : `check_ma` et `check_mi`. Ces noms suggèrent deux vérifications distinctes (majuscules et minuscules).

```
[0x00001060]> iz
nth paddr      vaddr      len size section type  string
―――――――
```

```c
    int count = 0;
    int i = 0;
    while (s[i] != '\0') {
        if (s[i] >= 'a' && s[i] <= 'z') {
            count++;
        }
        i++;
    }
    return count;
}

int main(int argc, char **argv)
{
    puts("---------------------");
    puts("----- CRACKME 8 -----");
    puts("---------------------");

    if (argc != 2) {
        puts("Missing  password !");
        return 1;
    }

    if (check_ma(argv[1]) == check_mi(argv[1])) {
        puts("** ACCESS GRANTED **");
    } else {
        puts("** ACCESS DENIED **");
    }
    return 0;
}
```